Amazon Bedrock AgentCore

Platform to build, connect, and optimize agents at scale

ATLASTICITY

aws

AWS MINNESOTA USER GROUP

September Meetup

Thursday, September 10, 2026

Welcome!

AGENDA

01 Welcome & Introductions

02 New AWS Community Group

03 Partner Introduction — Atlasticity

04 Workshop Access & Hands-On Lab

INTRODUCING

AWS Community Group

Led by

Farah Abdirahman

Why Join?

Connect

Network with local AWS builders and practitioners

Learn

Share knowledge and best practices with peers

Build

Collaborate on projects and hands-on labs

Grow

Advance your cloud skills and career

ATLASTICITY

aws

INTRODUCTIONS

Ryan Lindstedt
Senior Solutions Architect | Atlasticity

John Spaulding
Chief Technology Officer | Atlasticity

Max Anderson
Sr Solutions Architect | AWS

Paul DeLaria
Security Solution Architect | AWS

Norman Owens
Senior Solution Architect | AWS

ATLASTICITY

aws

AGENDA

- INTRODUCTIONS
- AWS AI PORTFOLIO OVERVIEW
- AGENTCORE OVERVIEW
- AGENTCORE CONCEPTS
- PUTTING IT ALL TOGETHER
- LAB PREP AND OVERVIEW
- LET’S BUILD!

ATLASTICITY

aws

WHO ATLASTICITY IS

Atlasticity is an AWS-focused Cloud & AI Operations partner

ADVISE

Work backward from business outcomes and risk

BUILD

Design and implement secure, resilient AWS solutions

OPERATE

Continuously optimize security, reliability, performance, and cost

EVOLVE

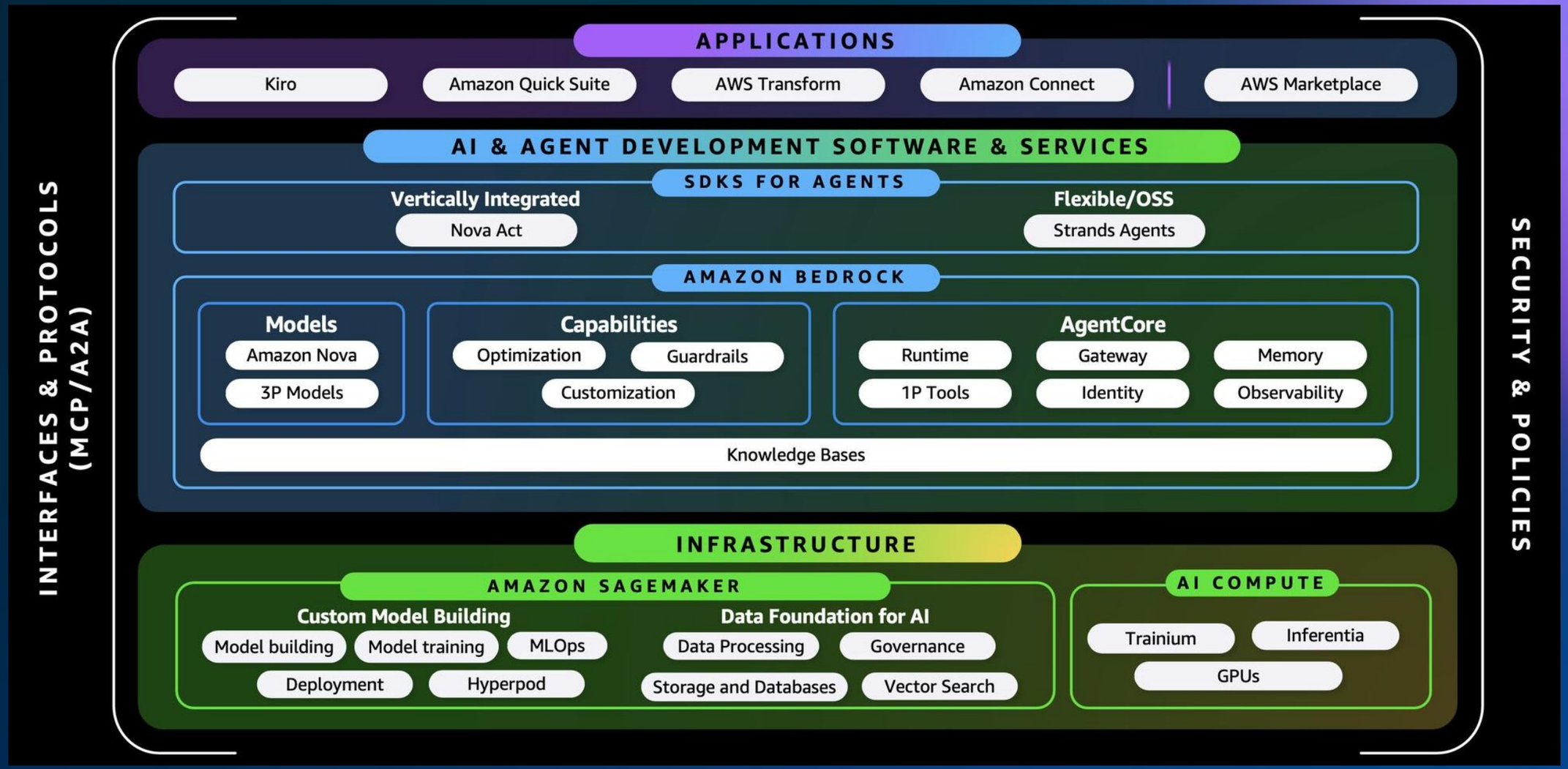
Modernize data and application foundations for responsible AI adoption

Security first • Outcome aligned • Built for continuous improvement

ATLASTICITY

aws

AWS AI PORTFOLIO



ATLASTICITY

aws

WHAT IS AWS AGENTCORE?

Amazon Bedrock

AgentCore

ATLASTICITY

aws

WHAT IS AGENTCORE?

A managed platform of production services for running AI agents — framework and model agnostic

Run the agent

Runtime hosts your agent code with session isolation and scaling, so you keep your framework and model choices.

Connect the tools

Gateway turns APIs and Lambda functions into MCP tools the agent can discover, with semantic search over the catalog.

Extend what it can do

Memory keeps short- and long-term context; managed Browser and Code Interpreter add sandboxed execution.

Govern and operate

Identity and Policy control who calls and what is allowed; Observability and Evaluations show what the agent actually did.

AgentCore is not a model and not an agent framework — it is the production layer around them. Adopt the services you need, individually.

ATLASTICITY

aws

WHY AGENTCORE?

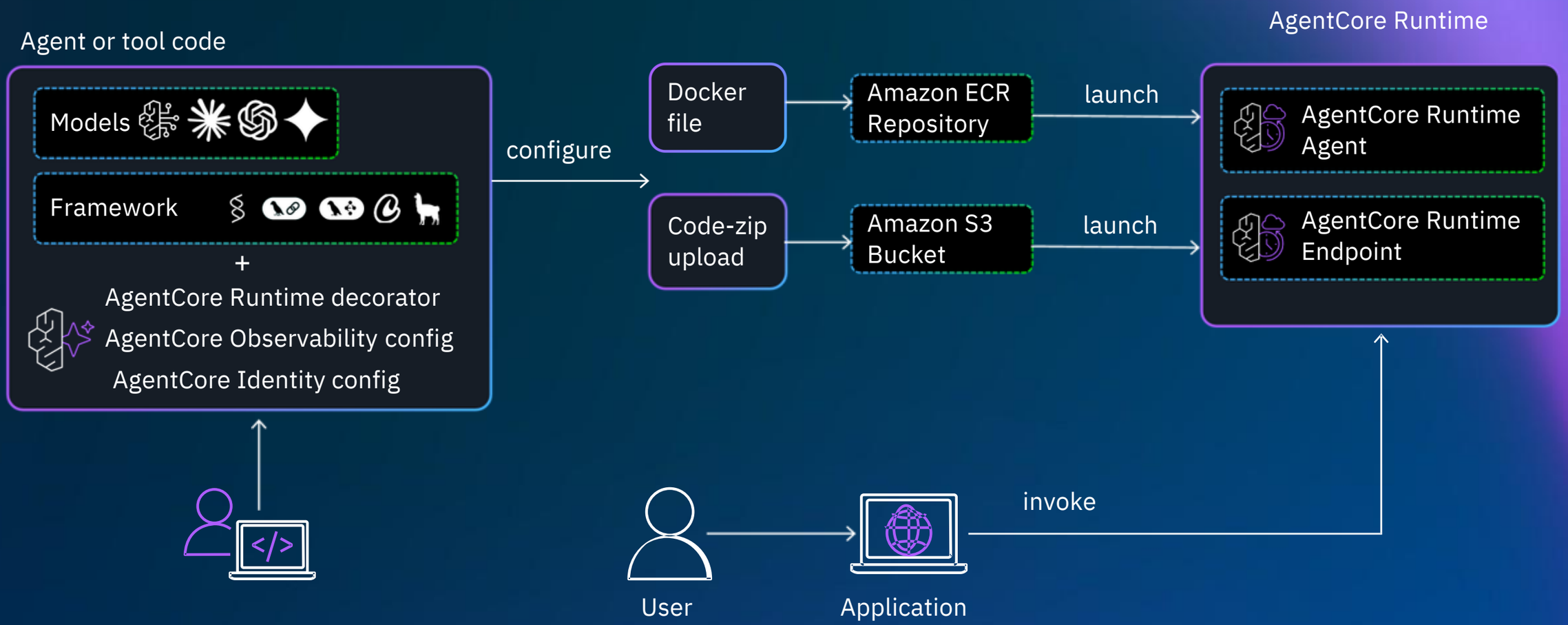
Prototype-to-Production Gap

"It works on my laptop" is the easy 20%. The hard 80% shows up when real users do.

ATLASTICITY

aws

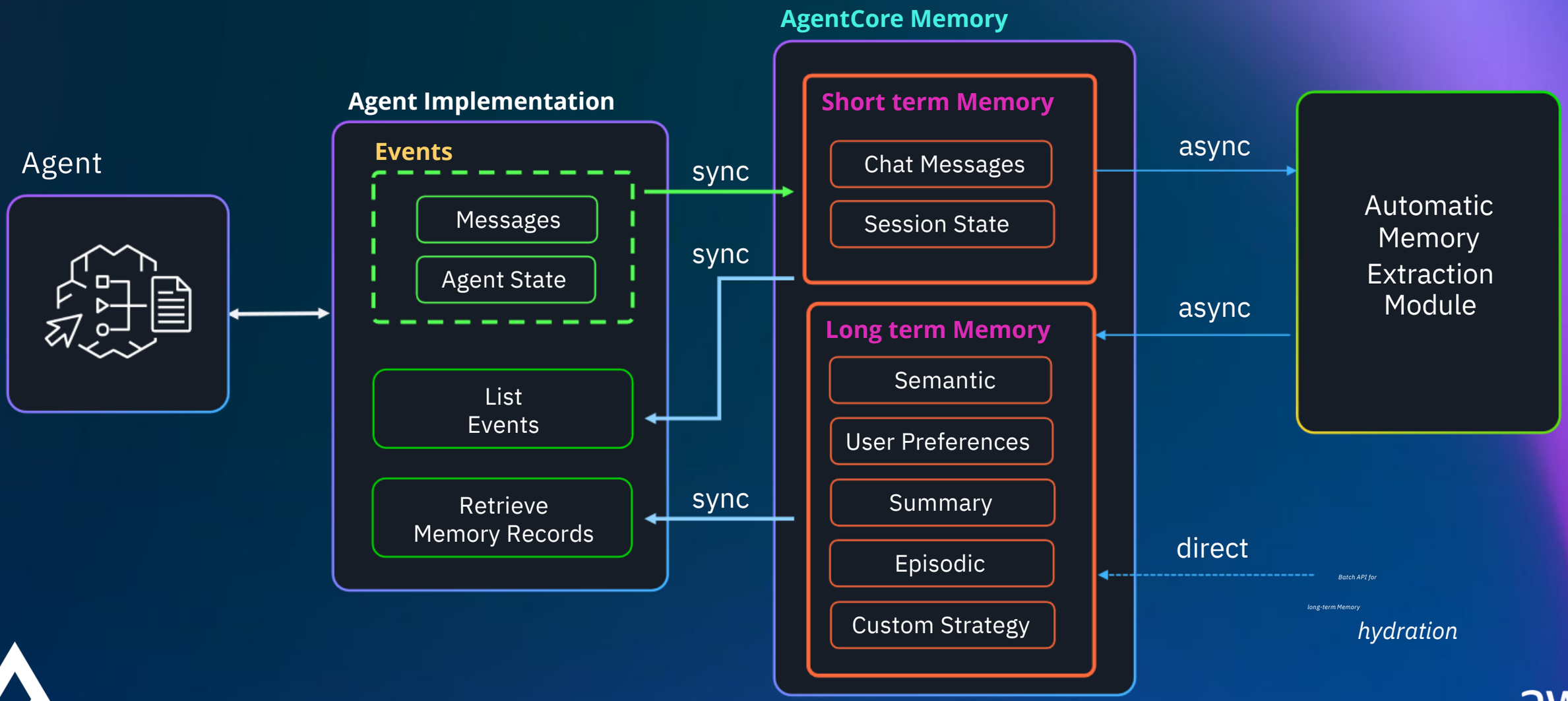
CONCEPT: AGENTCORE RUNTIME



ATLASTICITY

aws

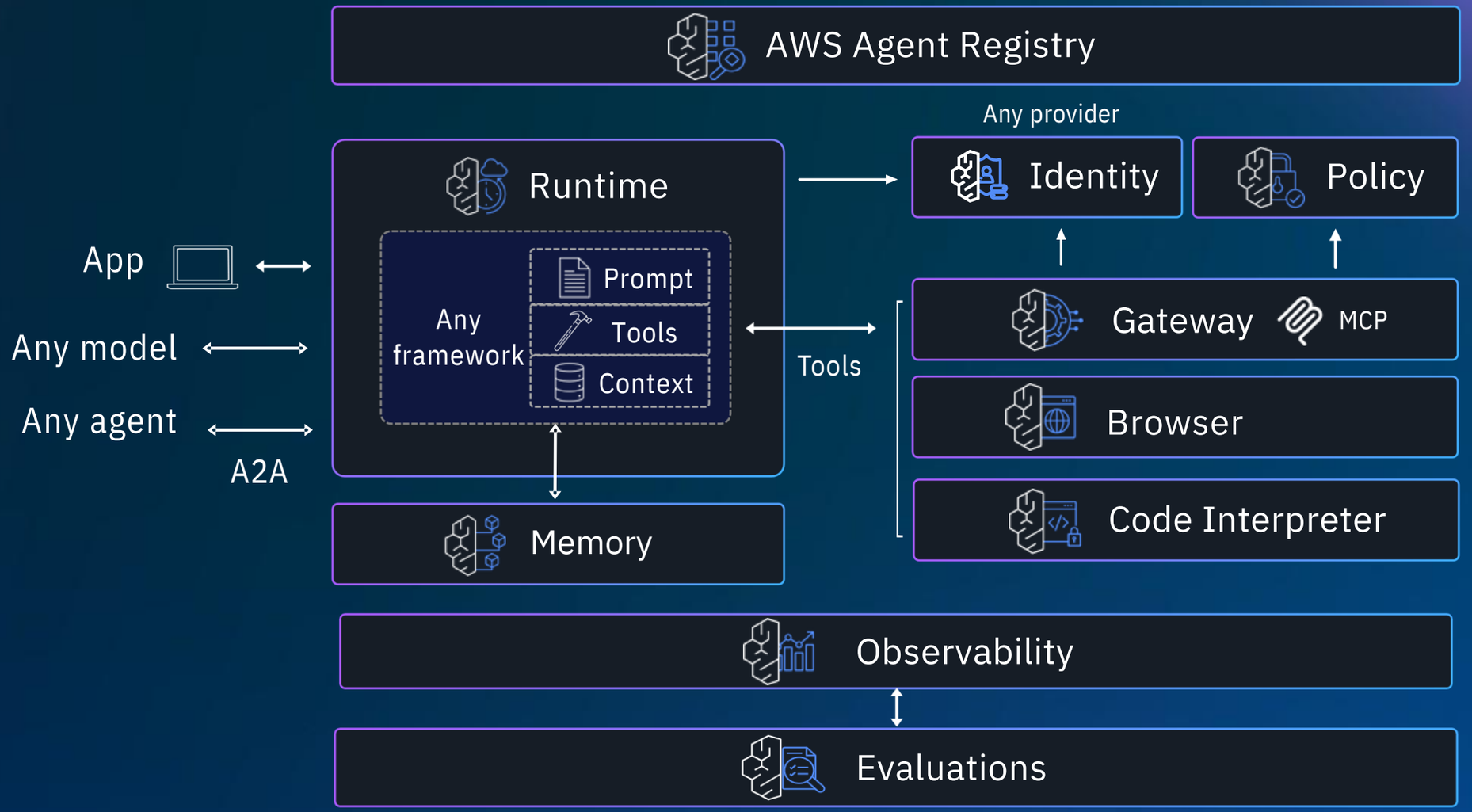
CONCEPT: AGENTCORE MEMORY



ATLASTICITY

aws

AGENTCORE BUILDING BLOCKS



ATLASTICITY

aws

PUTTING IT TOGETHER

Bedrock provides the models. AgentCore runs the agent. Strands is how you write it.

Strands Agents SDK

How you write the agent — model + prompt + tools, in a few lines of Python

open source · code-first · framework of choice for this lab

runs on

Amazon Bedrock AgentCore

The platform that deploys, runs, and operates the agent in production

- **Runtime** host & scale
- **Memory** context
- **Gateway** tools
- **Identity** auth
- **Observability** trace

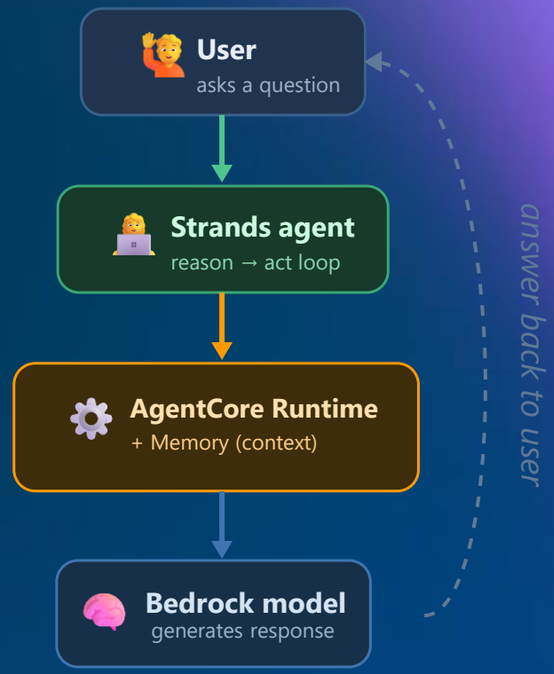
calls

Amazon Bedrock — Foundation Models

The reasoning "brains" the agent calls (e.g., Anthropic Claude, Amazon Nova)

- Claude
- Nova
- Llama
- any model

AT RUNTIME



Key idea: You write the agent with Strands, AgentCore runs it and gives it memory + tools, and it calls a Bedrock model to think.

Bedrock ≠ Bedrock AgentCore — one is the model service, the other is the agent platform.

ATLASTICITY

aws

THERE'S MORE: AGENTCORE HARNESS

Declare the agent in config — AgentCore runs the orchestration loop for you (GA June 2026)

The managed agent loop

You declare model, system prompt, tools, and memory. AgentCore runs the loop, compute, and state — no orchestration code to write.

Config, not framework code

Same AgentCore CLI, two paths: managed harness in declarative config files rather than a code-based agent in Strands or others

Tools are declarative

List what the agent calls: MCP endpoints, Gateway, Browser, Code Interpreter, or inline functions for human-in-the-loop.

Stateful and swappable

Each session is an isolated microVM with its own filesystem, shell, and persistent memory. Swap models mid-session; override defaults per invocation.

AgentCore Harness automates the plumbing, not the architecture — Gateway, Identity, Policy, and Observability still define your trust boundary.

ATLASTICITY

aws

WHAT WE'LL BUILD AND WHY

Lab 1: Building the Agent Prototype
Lab 2: Add Memory to Your Agent
Lab 3: Scaling Tools with Gateway
Lab 4: Securing and Observing in Production

Lab 5: Evaluating Agent Quality
Lab 6: Building the Customer Interface
Lab 7: Governing Agent Actions with Policies
Lab 8: Zero-Code Agents with AgentCore Harness
Lab 9: Optimizing Agent Quality

02

AgentCore Runtime

Build a simple agent and deploy it to a serverless, production-ready endpoint.

- ✓ Deploy in minutes, not weeks
- ✓ Invoke it like a real service
- ✓ Auto-scaling & session isolation, built in
- ✓ No servers to manage

03

AgentCore Memory

Give the agent memory so it remembers context within and across sessions.

- ✓ Short-term: multi-turn conversation
- ✓ Long-term: preferences & facts persist

WHY THIS MATTERS

Runtime and Memory are the first two problems you hit taking ANY agent to production.

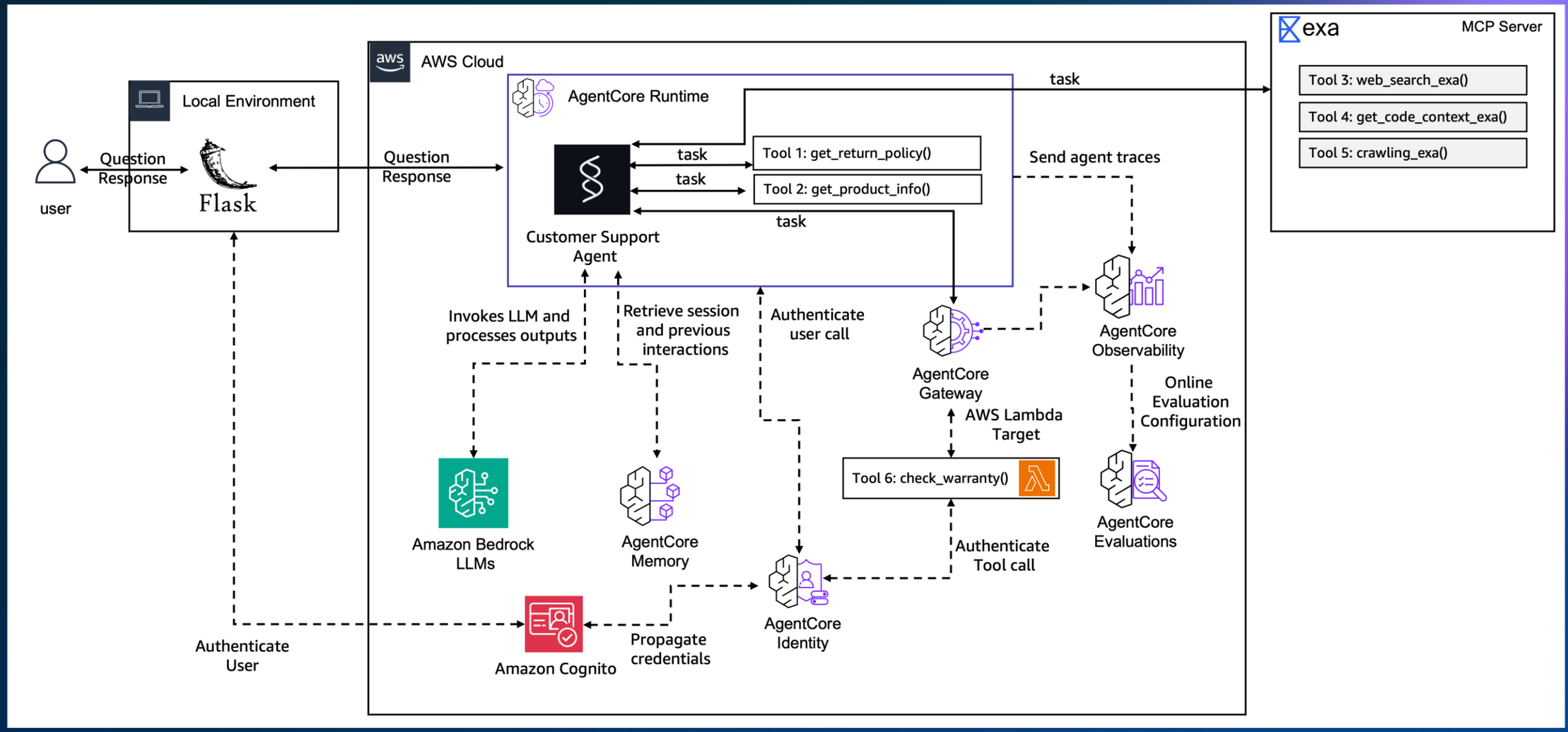
Get these right and the rest of AgentCore — Gateway, Identity, Observability — slots in the same way.

A simple agent, deployed to a serverless endpoint, that remembers you across sessions.

ATLASTICITY

aws

ENTIRE LAB DIAGRAM



ATLASTICITY

aws

LAB TIPS, LINKS, & MORE

https://atlasticity.github.io/AWS_MN/

ATLASTICITY

aws

Let's Build!